# PYTHON ACTIVITY-BASED LEARNING

# 10 Randomly Selected Case Studies

Includes problem statements, objectives, assumptions, algorithms, pseudocode, flowcharts, Python source code, test cases, sample output screenshots, and conclusions.

Original case study numbers: 42, 7, 29, 51, 16, 38, 63, 26, 9, 75

Prepared for B.Tech Python Programming activity-based learning.

Note: Flowcharts and console images are illustrative representations generated for this document. Run the included code in Python to capture your own execution screenshots if your faculty requires actual IDE screenshots.

# Contents

1. Case Study 42 — Student Marks Matrix Analysis
2. Case Study 7 — Retail Billing with Multiple Discount Rules
3. Case Study 29 — Password Policy Validation Engine
4. Case Study 51 — Bus Passenger Occupancy Simulation
5. Case Study 16 — Multi-Day Water Tank Simulation
6. Case Study 38 — ATM Menu-Driven Banking Simulation
7. Case Study 63 — Multi-Product Price Fluctuation Analysis
8. Case Study 26 — Nested Loop Examination Seating Generator
9. Case Study 9 — Multi-Currency Travel Budget Analysis
10. Case Study 75 — Integrated Student Record Consistency Checker

# Case Study 42 — Student Marks Matrix Analysis

**Python concepts:** Nested lists, loops, averages, row-wise and column-wise statistics

## 1. Problem Statement

Analyse marks for several students across multiple subjects. Calculate each student's total and average, then calculate the average marks for each subject.

## 2. Objectives

• Store marks in a nested list.

• Calculate student totals and averages.

• Calculate subject-wise averages.

## 3. Assumptions

• Each row represents one student and each column represents one subject.

• All marks are numeric and use the same maximum-mark scale.

## 4. Algorithm

1. Start.

2. Store the marks in a nested list.

3. For each student row, calculate the total and average and display them.

4. For each subject column, add the marks of all students and divide by the number of students.

5. Display each subject average.

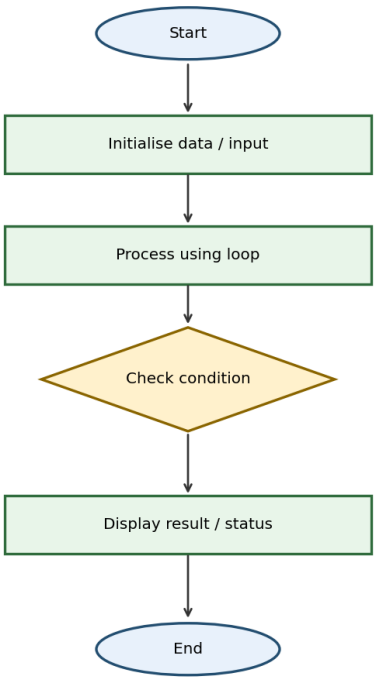
6. Stop.

## 5. Pseudocode

```
START
marks ← [[85, 90, 78], [72, 80, 88], [95, 92, 96]]
FOR each student row in marks
    total ← sum of row
    average ← total / number of subjects
    DISPLAY total and average
FOR each subject column
    total ← sum of that column across all students
    DISPLAY total / number of students
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 42: Student Marks Matrix Analysis

## 7. Python Source Code

```
marks = [
    [85, 90, 78],
    [72, 80, 88],
    [95, 92, 96]
]

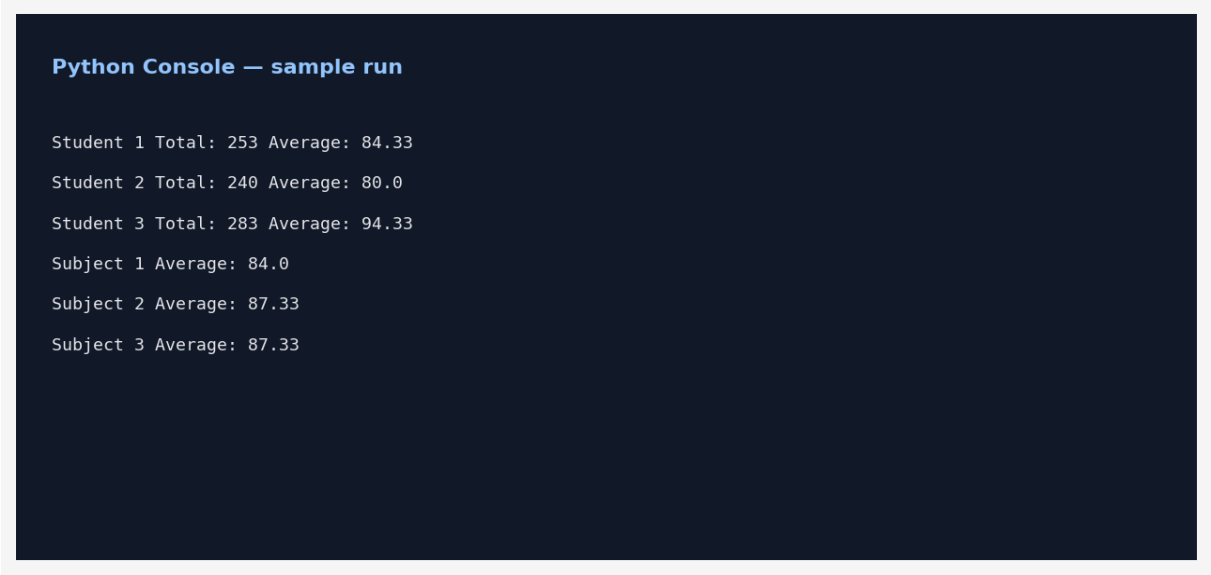
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print("Student", i + 1, "Total:", total,
          "Average:", round(average, 2))

for j in range(len(marks[0])):
    total = 0
    for i in range(len(marks)):
        total += marks[i][j]
    print("Subject", j + 1, "Average:",
          round(total / len(marks), 2))
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Student 1 marks: 85, 90, 78 | Total 253; Average 84.33 |
| Student 2 marks: 72, 80, 88 | Total 240; Average 80.00 |
| Subject 1 marks: 85, 72, 95 | Average 84.00 |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

Nested lists make it convenient to organise marks in rows and columns and calculate both student-wise and subject-wise statistics.

# Case Study 7 — Retail Billing with Multiple Discount Rules

**Python concepts:** Lists, loops, arithmetic operators, if-else

## 1. Problem Statement

Generate an itemised retail bill from product names, unit prices, and quantities. Apply a discount based on the bill subtotal.

## 2. Objectives

• Store product details in lists.

• Calculate each item's amount and subtotal.

• Apply a threshold-based discount and display the final bill.

## 3. Assumptions

• A 10% discount applies to subtotals of ■1,000 or more; otherwise, a 5% discount applies.

• Tax is not included in this simplified example.

## 4. Algorithm

1. Start.

2. Store product names, prices and quantities in lists.

3. Set subtotal to zero.

4. For each product, calculate price × quantity and add it to the subtotal.

5. If subtotal is at least ■1,000, set discount to 10%; otherwise set it to 5%.

6. Calculate final amount and display the bill.

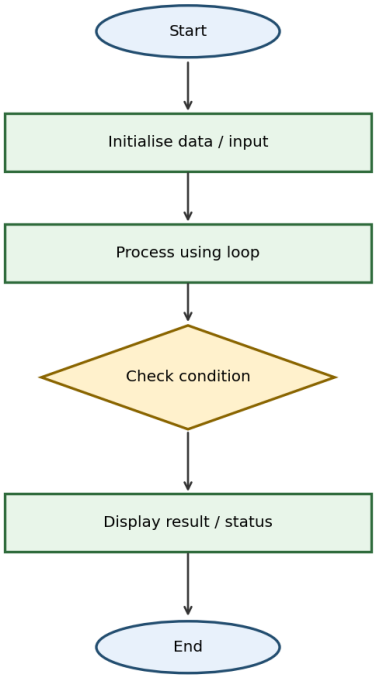
7. Stop.

## 5. Pseudocode

```
START
products ← [Book, Bag, Pen]
prices ← [200, 800, 20]
quantities ← [2, 1, 5]
subtotal ← 0
FOR each product index i
    amount ← prices[i] × quantities[i]
    DISPLAY product and amount
    subtotal ← subtotal + amount
IF subtotal ≥ 1000 THEN discount ← subtotal × 0.10
ELSE discount ← subtotal × 0.05
final ← subtotal – discount
DISPLAY subtotal, discount, final
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 7: Retail Billing with Multiple Discount Rules

## 7. Python Source Code

```
products = ["Book", "Bag", "Pen"]
prices = [200, 800, 20]
quantities = [2, 1, 5]

subtotal = 0
print("----- RETAIL BILL -----")

for i in range(len(products)):
    amount = prices[i] * quantities[i]
    print(products[i], "=", amount)
    subtotal += amount

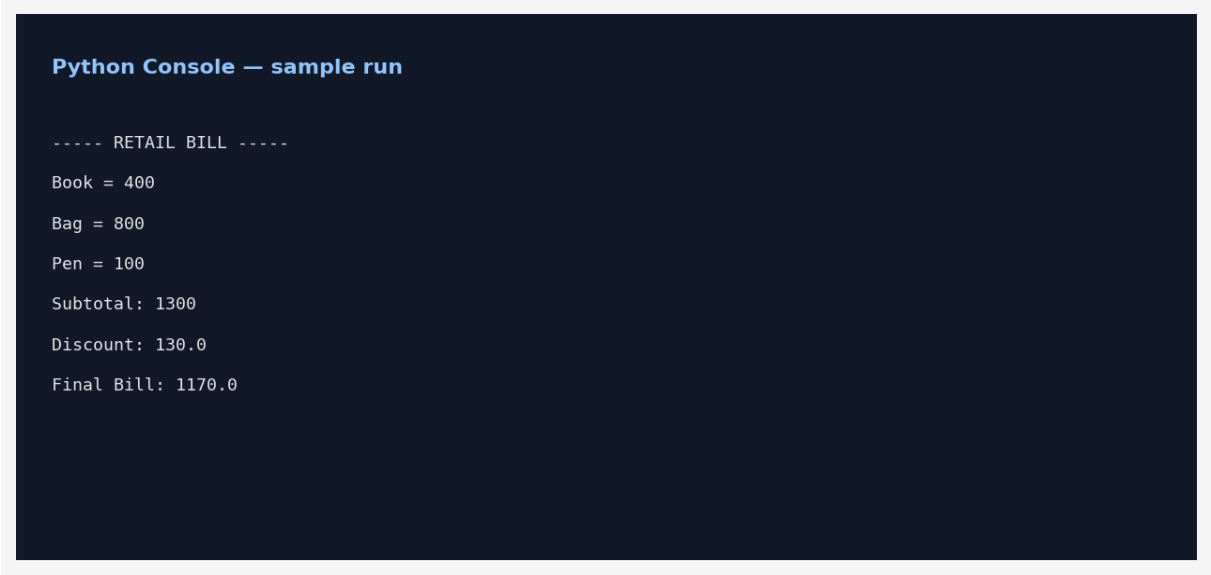
if subtotal >= 1000:
    discount = subtotal * 0.10
else:
    discount = subtotal * 0.05

final_amount = subtotal - discount
print("Subtotal:", subtotal)
print("Discount:", round(discount, 2))
print("Final Bill:", round(final_amount, 2))
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Book: ■200 × 2 | ■400 |
| Bag: ■800 × 1 | ■800 |
| Whole bill: Subtotal ■1,300 | 10% discount ■130; final ■1,170 |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

Lists and a loop support item-wise billing, while an if-else statement applies the correct discount rule.

# Case Study 29 — Password Policy Validation Engine

**Python concepts:** Strings, loops, Boolean variables, conditional statements

## 1. Problem Statement

Check a sample password against basic rules: at least eight characters, at least one digit, and at least one uppercase letter. Report failed rules.

## 2. Objectives

• Examine characters in a string.

• Check length and character categories.

• Display clear validation messages.

## 3. Assumptions

• This is an educational example and is not a complete real-world password security checker.

• A password is considered valid only when all three rules pass.

## 4. Algorithm

1. Start.

2. Read a password.

3. Set digit-found and uppercase-found flags to false.

4. Check each character and update the flags.

5. Check length, digit flag and uppercase flag.

6. Display valid if all conditions pass; otherwise display each failed condition.

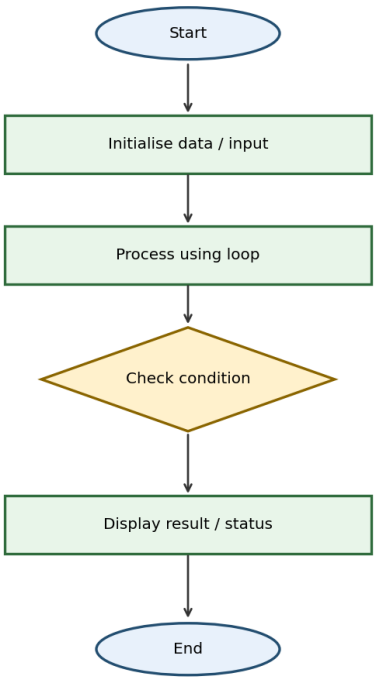
7. Stop.

## 5. Pseudocode

```
START
INPUT password
has_digit ← FALSE
has_upper ← FALSE
FOR each character ch in password
    IF ch is a digit THEN has_digit ← TRUE
    IF ch is uppercase THEN has_upper ← TRUE
IF length(password) ≥ 8 AND has_digit AND has_upper
    DISPLAY Valid
ELSE
    DISPLAY Invalid and each unmet rule
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 29: Password Policy Validation Engine

## 7. Python Source Code

```
password = input("Enter password: ")

has_digit = False
has_upper = False

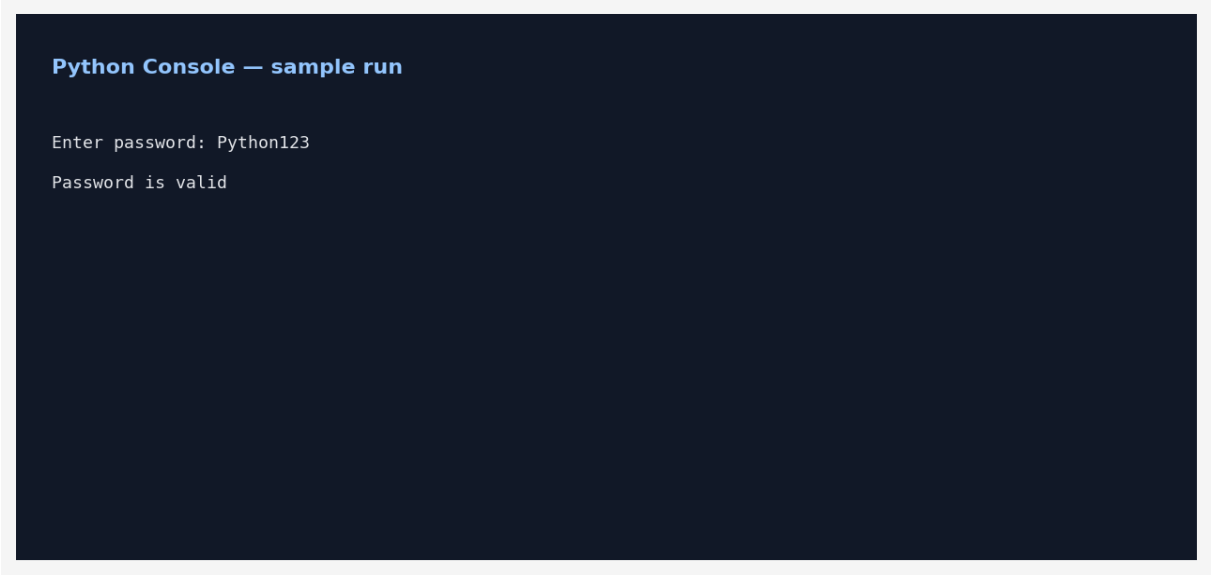
for ch in password:
    if ch.isdigit():
        has_digit = True
    if ch.isupper():
        has_upper = True

if len(password) >= 8 and has_digit and has_upper:
    print("Password is valid")
else:
    print("Password is invalid")
    if len(password) < 8:
        print("Minimum 8 characters required")
    if not has_digit:
        print("Include at least one digit")
    if not has_upper:
        print("Include at least one uppercase letter")
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Input: Python123 | Valid |
| Input: python123 | Invalid: uppercase letter missing |
| Input: Py12 | Invalid: minimum length not met |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

String methods and Boolean flags can validate simple password rules and explain which conditions are not satisfied.

# Case Study 51 — Bus Passenger Occupancy Simulation

**Python concepts:** Lists, loops, arithmetic, if-else

## 1. Problem Statement

Track the number of passengers on a bus at each stop using boarding and alighting counts, and check whether the bus exceeds its capacity.

## 2. Objectives

• Update passenger counts at every stop.

• Display occupancy after each stop.

• Flag capacity violations.

## 3. Assumptions

• The bus starts empty.

• Alighting counts never exceed the number of passengers already on board.

• Bus capacity is 50 passengers.

## 4. Algorithm

1. Start with zero passengers and capacity 50.

2. Store boarding and alighting counts for each stop.

3. For each stop, add boarding passengers and subtract alighting passengers.

4. Display the passenger count.

5. Compare the count with capacity and display the status.

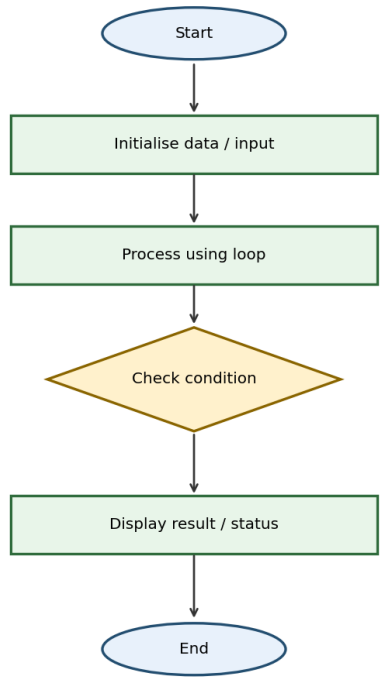
6. Stop.

## 5. Pseudocode

```
START
capacity ← 50
passengers ← 0
boarding ← [15, 20, 10, 8]
alighting ← [0, 5, 12, 3]
FOR each stop i
    passengers ← passengers + boarding[i]
    passengers ← passengers – alighting[i]
    DISPLAY stop and passengers
    IF passengers > capacity THEN DISPLAY Capacity exceeded
    ELSE DISPLAY Within capacity
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 51: Bus Passenger Occupancy Simulation

## 7. Python Source Code

```
capacity = 50
passengers = 0
boarding = [15, 20, 10, 8]
alighting = [0, 5, 12, 3]

for i in range(len(boarding)):
    passengers += boarding[i]
    passengers -= alighting[i]

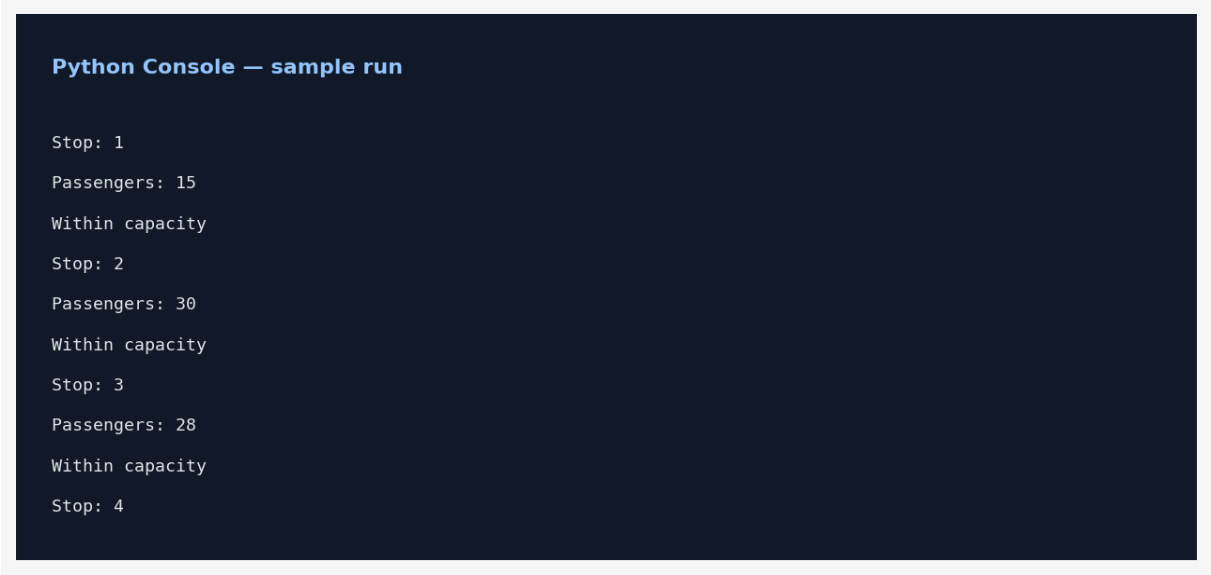
    print("Stop:", i + 1)
    print("Passengers:", passengers)

    if passengers > capacity:
        print("Capacity exceeded")
    else:
        print("Within capacity")
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Stop 1: Board 15, alight 0 | 15 passengers; within capacity |
| Stop 2: Board 20, alight 5 | 30 passengers; within capacity |
| Stop 4: Board 8, alight 3 | 33 passengers; within capacity |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

A running total and a loop simulate passenger occupancy across multiple stops and detect capacity violations.

# Case Study 16 — Multi-Day Water Tank Simulation

**Python concepts:** Variables, lists, loops, conditional statements

## 1. Problem Statement

Simulate a tank's water level over several days based on inflow and outflow. Report overflow or shortage and keep the level within the tank's capacity.

## 2. Objectives

• Update water level day by day.

• Detect overflow and shortage.

• Display the water level after each day.

## 3. Assumptions

• Tank capacity is 1,000 litres and the initial level is 500 litres.

• If the calculated level exceeds capacity, excess water is discarded; if below zero, the level is set to zero.

## 4. Algorithm

1. Start with capacity 1,000 and water level 500.

2. Store daily inflow and outflow values.

3. For each day, add inflow and subtract outflow.

4. If the level exceeds capacity, report overflow and set it to capacity.

5. If the level is below zero, report shortage and set it to zero.

6. Display the final level for that day.

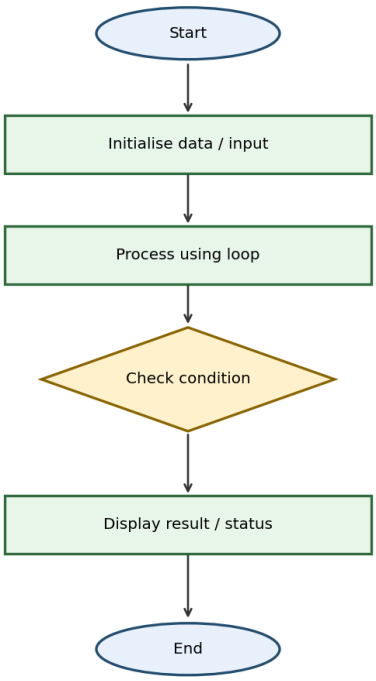
7. Stop.

## 5. Pseudocode

```
START
capacity ← 1000
water ← 500
inflow ← [200, 300, 100, 250]
outflow ← [100, 150, 200, 100]
FOR each day i
    water ← water + inflow[i] − outflow[i]
    IF water > capacity THEN DISPLAY Overflow; water ← capacity
    ELSE IF water < 0 THEN DISPLAY Shortage; water ← 0
    DISPLAY day and water
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 16: Multi-Day Water Tank Simulation

## 7. Python Source Code

```
capacity = 1000
water = 500
inflow = [200, 300, 100, 250]
outflow = [100, 150, 200, 100]

for day in range(len(inflow)):
    water = water + inflow[day] - outflow[day]

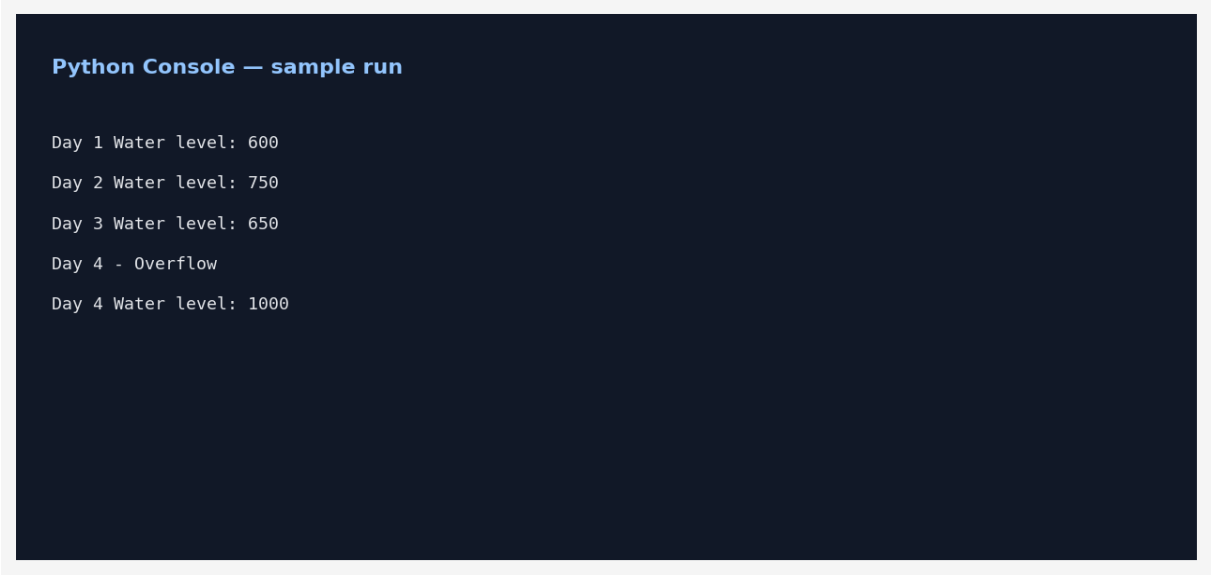
    if water > capacity:
        print("Day", day + 1, "- Overflow")
        water = capacity
    elif water < 0:
        print("Day", day + 1, "- Water shortage")
        water = 0

    print("Day", day + 1, "Water level:", water)
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Day 1: 500 + 200 – 100 | 600 litres |
| Day 2: 600 + 300 – 150 | 750 litres |
| Day 4: 850 + 250 – 100 | 1,000 litres after overflow adjustment |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

The simulation demonstrates how loops and conditions can track a changing quantity while enforcing a fixed capacity.

# Case Study 38 — ATM Menu-Driven Banking Simulation

**Python concepts:** While loop, input, conditions, arithmetic

## 1. Problem Statement

Create a simple ATM menu that allows deposits, withdrawals, balance enquiries, and exit. Reject non-positive amounts and withdrawals exceeding the balance.

## 2. Objectives

• Repeat menu options until exit.

• Update the account balance correctly.

• Validate deposits and withdrawals.

## 3. Assumptions

• The opening balance is ■5,000.

• This is a local educational simulation and does not connect to a real bank.

• Only positive deposit and withdrawal amounts are accepted.

## 4. Algorithm

1. Start with balance ■5,000.

2. Display the menu repeatedly.

3. Read the user's choice.

4. For deposit, accept a positive amount and add it to the balance.

5. For withdrawal, accept a positive amount not exceeding the balance and subtract it.

6. For balance enquiry, display the balance.

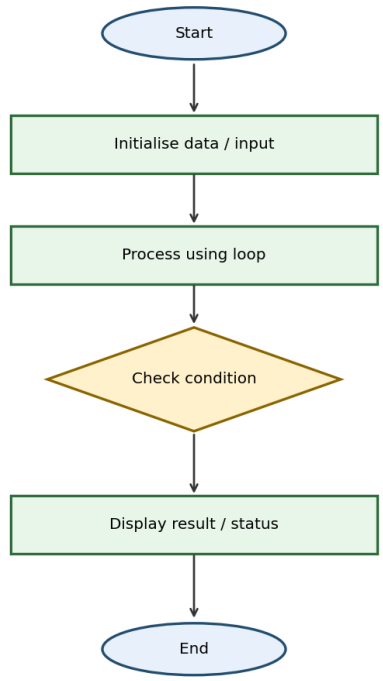
7. For exit, display a message and stop; otherwise report an invalid choice.

## 5. Pseudocode

```
START
balance ← 5000
REPEAT
    DISPLAY menu: Deposit, Withdraw, Balance, Exit
    INPUT choice
    IF choice = 1: INPUT amount; IF amount > 0 THEN add to balance
    ELSE IF choice = 2: INPUT amount; check amount > 0 and amount ≤ balance
        IF valid THEN subtract amount ELSE DISPLAY error
    ELSE IF choice = 3: DISPLAY balance
    ELSE IF choice = 4: DISPLAY Thank you; EXIT LOOP
    ELSE DISPLAY Invalid choice
UNTIL exit
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 38: ATM Menu-Driven Banking Simulation

## 7. Python Source Code

```
balance = 5000

while True:
    print("\\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        amount = float(input("Enter deposit amount: "))
        if amount > 0:
            balance += amount
            print("Deposit successful")
        else:
            print("Enter a positive amount")

    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))
        if amount > 0 and amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Invalid amount or insufficient balance")

    elif choice == "3":
        print("Balance:", balance)

    elif choice == "4":
        print("Thank you!")
        break

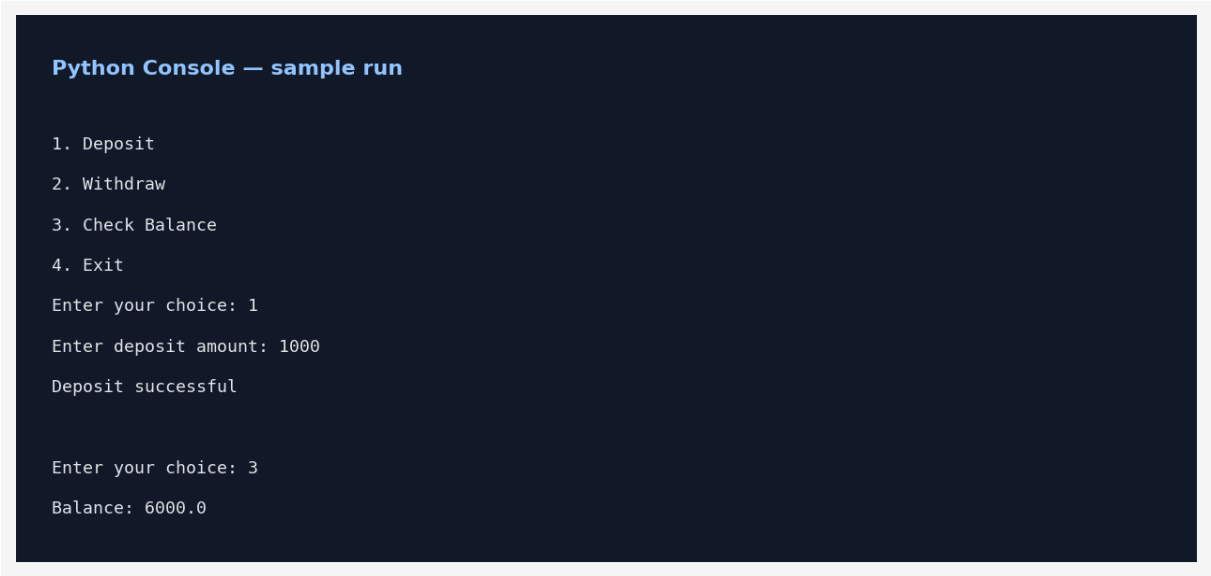
    else:
        print("Invalid choice")
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Choice 1; amount 1000: Deposit | Balance becomes ■6,000 |

| Test / Input | Expected result |
|---|---|
| Choice 2; amount 500: Withdraw | Balance becomes ■5,500 |
| Choice 2; amount 7000: Withdraw | Rejected: insufficient balance |
| Choice 4: Exit | Thank you! |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

A while loop provides a repeated menu, and conditional statements select operations and validate transactions.

# Case Study 63 — Multi-Product Price Fluctuation Analysis

**Python concepts:** Lists, arithmetic operators, percentages, if-elif-else

## 1. Problem Statement

Compare old and new prices for several products, calculate percentage change, and classify each price as increased, decreased, or unchanged.

## 2. Objectives

• Store old and new prices in lists.

• Calculate percentage price changes.

• Classify changes using conditions.

## 3. Assumptions

• All old prices are greater than zero to avoid division by zero.

• Prices are sample values in the same currency.

## 4. Algorithm

1. Start.

2. Store product names, old prices and new prices.

3. For each product, calculate new price minus old price.

4. Calculate percentage change using the old price as the base.

5. Display the percentage change.

6. Classify the price as increased, decreased or unchanged.

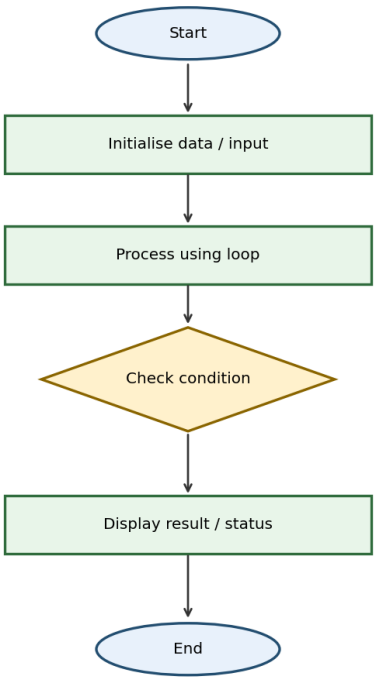
7. Stop.

## 5. Pseudocode

```
START
products ← [Laptop, Phone, Tablet]
old_prices ← [50000, 20000, 15000]
new_prices ← [55000, 18000, 16500]
FOR each product index i
    change ← new_prices[i] − old_prices[i]
    percentage ← change / old_prices[i] × 100
    DISPLAY product and percentage
    IF percentage > 0 DISPLAY Increased
    ELSE IF percentage < 0 DISPLAY Decreased
    ELSE DISPLAY No change
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 63: Multi-Product Price Fluctuation Analysis

## 7. Python Source Code

```
products = ["Laptop", "Phone", "Tablet"]
old_prices = [50000, 20000, 15000]
new_prices = [55000, 18000, 16500]

for i in range(len(products)):
    change = new_prices[i] - old_prices[i]
    percentage = change / old_prices[i] * 100

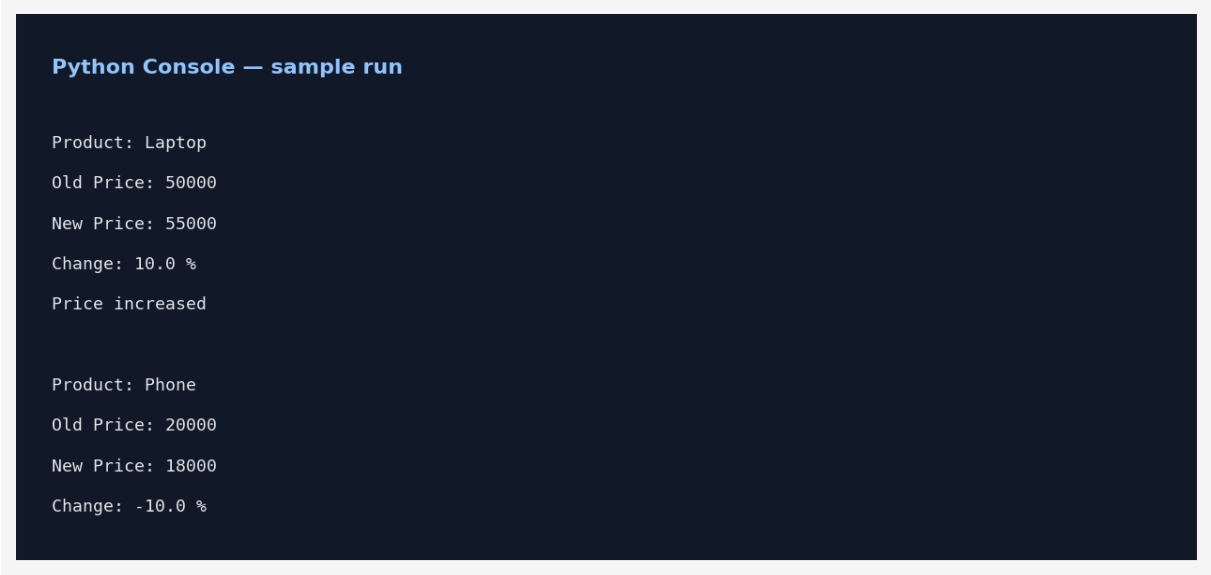
    print("\\nProduct:", products[i])
    print("Old Price:", old_prices[i])
    print("New Price:", new_prices[i])
    print("Change:", round(percentage, 2), "%")

    if percentage > 0:
        print("Price increased")
    elif percentage < 0:
        print("Price decreased")
    else:
        print("No price change")
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Laptop: ■50,000 → ■55,000 | +10%; increased |
| Phone: ■20,000 → ■18,000 | –10%; decreased |
| Tablet: ■15,000 → ■16,500 | +10%; increased |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

Percentage change provides a fair comparison between products with different original prices.

# Case Study 26 — Nested Loop Examination Seating Generator

**Python concepts:** Nested loops, tuples, conditions, pattern generation

## 1. Problem Statement

Print a classroom seating grid and mark one specified seat as reserved.

## 2. Objectives

• Generate rows and columns using nested loops.

• Represent a reserved seat as a tuple.

• Display a readable seating arrangement.

## 3. Assumptions

• The classroom has four rows and five columns.

• Seat coordinates are 1-based; seat (2, 3) is reserved.

• S means available seat and R means reserved seat.

## 4. Algorithm

1. Start.

2. Set the number of rows and columns.

3. Store the reserved seat as a tuple.

4. Use an outer loop for each row and an inner loop for each column.

5. If the current coordinate matches the reserved tuple, print R; otherwise print S.

6. Start a new output line after each row.

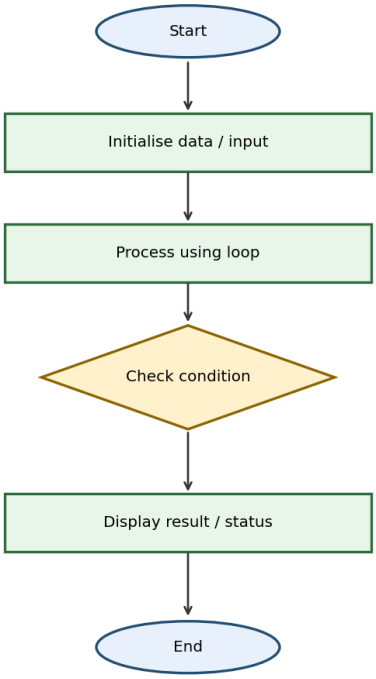
7. Stop.

## 5. Pseudocode

```
START
rows ← 4; columns ← 5
reserved_seat ← (2, 3)
FOR row from 1 to rows
    FOR column from 1 to columns
        IF (row, column) = reserved_seat THEN DISPLAY R
        ELSE DISPLAY S
    DISPLAY new line
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 26: Nested Loop Examination Seating Generator

## 7. Python Source Code

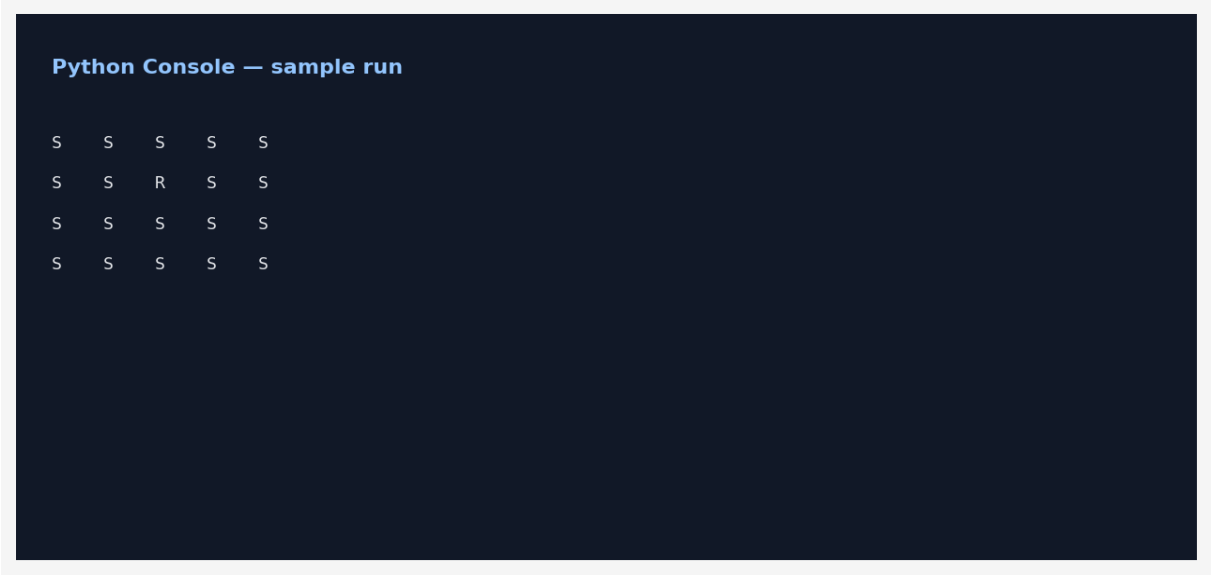
```
rows = 4
columns = 5
reserved_seat = (2, 3)

for i in range(1, rows + 1):
    for j in range(1, columns + 1):
        if (i, j) == reserved_seat:
            print("R", end="\\t")
        else:
            print("S", end="\\t")
    print()
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Seat (1, 1): Not reserved | S |
| Seat (2, 3): Reserved coordinate | R |
| Seat (4, 5): Not reserved | S |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

Nested loops generate the seating grid, while a tuple conveniently stores the reserved seat's row and column.

# Case Study 9 — Multi-Currency Travel Budget Analysis

**Python concepts:** Lists, loops, arithmetic, percentages, conditions

## 1. Problem Statement

Convert travel expenses from a foreign currency to rupees, add a conversion fee, compare the total with a budget, and show the remaining budget or overspend.

## 2. Objectives

• Store expenses by category.

• Convert expenses using an example exchange rate.

• Include a conversion fee and compare with the budget.

## 3. Assumptions

• The sample exchange rate is ■83 per foreign currency unit.

• A 2% conversion fee applies to each converted expense.

• Budget is ■30,000; sample expenses are 200, 100 and 50 foreign currency units.

## 4. Algorithm

1. Start.

2. Store expense categories and foreign-currency amounts.

3. Set exchange rate to 83, fee rate to 2%, and budget to ■30,000.

4. For each category, convert the amount to rupees and calculate the fee.

5. Add the converted amount and fee to the total.

6. Compare total with budget and display budget status and difference.

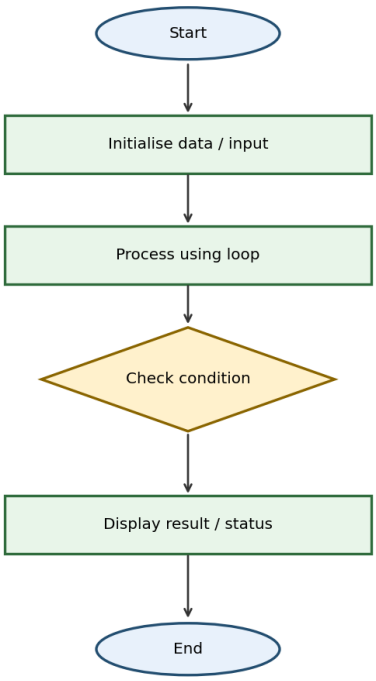
7. Stop.

## 5. Pseudocode

```
START
categories ← [Hotel, Food, Transport]
expenses ← [200, 100, 50]
rate ← 83; fee_rate ← 0.02; budget ← 30000
total ← 0
FOR each expense i
    rupees ← expenses[i] × rate
    fee ← rupees × fee_rate
    cost ← rupees + fee
    total ← total + cost
    DISPLAY category and cost
IF total ≤ budget DISPLAY Within budget
ELSE DISPLAY Budget exceeded
DISPLAY budget − total
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 9: Multi-Currency Travel Budget Analysis

## 7. Python Source Code

```
categories = ["Hotel", "Food", "Transport"]
expenses = [200, 100, 50]

exchange_rate = 83
conversion_fee = 0.02
budget = 30000
total = 0

for i in range(len(categories)):
    rupees = expenses[i] * exchange_rate
    fee = rupees * conversion_fee
    cost = rupees + fee
    total += cost
    print(categories[i], "Cost:", round(cost, 2), "INR")

print("Total Expense:", round(total, 2), "INR")

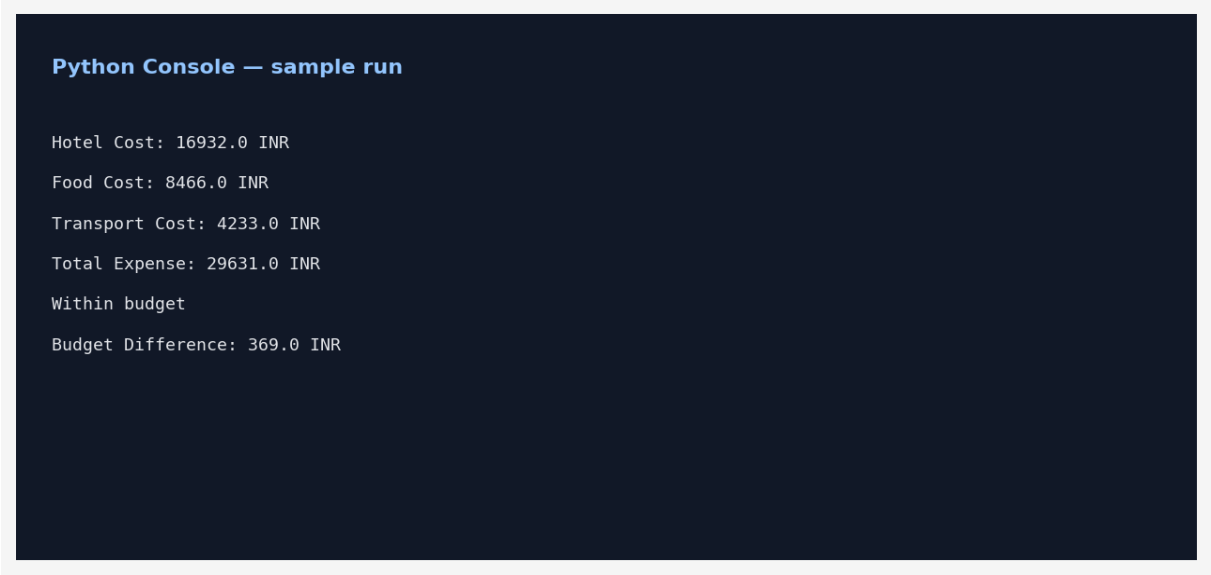
if total <= budget:
    print("Within budget")
else:
    print("Budget exceeded")

print("Budget Difference:", round(budget - total, 2), "INR")
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| Hotel: 200 × ■83 plus 2% fee | ■16,932.00 |
| Food: 100 × ■83 plus 2% fee | ■8,466.00 |
| All expenses: Total ■29,631 | Within ■30,000 budget; ■369 left |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

Lists and arithmetic operations help organise travel expenses, convert currencies, include fees and check a spending limit.

# Case Study 75 — Integrated Student Record Consistency Checker

**Python concepts:** Tuples, lists, strings, conditions, duplicate detection

## 1. Problem Statement

Validate student records containing roll number, name, marks and attendance. Detect duplicate roll numbers, missing names, and out-of-range marks or attendance.

## 2. Objectives

• Store records as tuples in a list.

• Detect duplicate identifiers.

• Validate names, marks and attendance ranges.

## 3. Assumptions

• Marks and attendance are percentages from 0 to 100.

• Roll numbers should be unique.

• A record is valid only if all four checks pass.

## 4. Algorithm

1. Start.

2. Store student records as tuples in a list.

3. Create an empty list of seen roll numbers.

4. For each record, unpack roll number, name, marks and attendance.

5. Check whether the roll number has already appeared; report duplicates.

6. Check for missing names and values outside 0–100.

7. Report valid only if no validation error exists; add the roll number to the seen list.

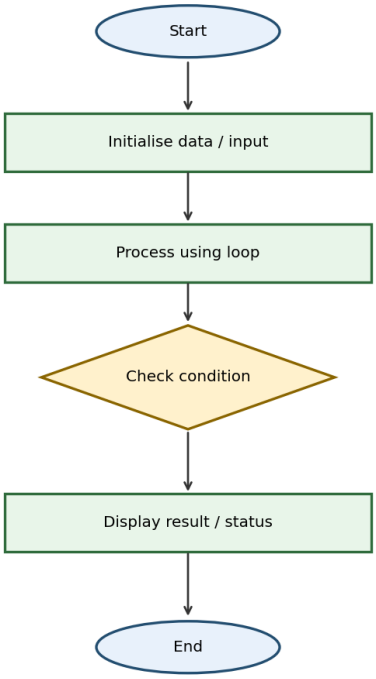
8. Stop.

## 5. Pseudocode

```
START
records ← list of tuples (roll, name, marks, attendance)
seen_ids ← empty list
FOR each record
    UNPACK roll, name, marks, attendance
    error ← FALSE
    IF roll in seen_ids THEN DISPLAY duplicate ID; error ← TRUE
    IF name is empty THEN DISPLAY missing name; error ← TRUE
    IF marks < 0 OR marks > 100 THEN DISPLAY invalid marks; error ← TRUE
    IF attendance < 0 OR attendance > 100 THEN DISPLAY invalid attendance; error ←
           TRUE
    IF error = FALSE THEN DISPLAY Record valid
    IF roll not in seen_ids THEN append roll to seen_ids
STOP
```

## 6. Flowchart

Simplified flowchart for the main program logic:




Case Study 75: Integrated Student Record Consistency Checker

## 7. Python Source Code

```
students = [
    ("101", "Ravi", 85, 90),
    ("102", "Sita", 78, 82),
    ("101", "Arun", 95, 88),
    ("104", "", 110, 75)
]

ids = []

for student in students:
    roll, name, marks, attendance = student
    errors = []

    if roll in ids:
        errors.append("Duplicate ID")
    if name == "":
        errors.append("Missing name")
    if marks < 0 or marks > 100:
        errors.append("Invalid marks")
    if attendance < 0 or attendance > 100:
        errors.append("Invalid attendance")

    if errors:
        print(roll, name, "->", ", ".join(errors))
    else:
        print(roll, name, "-> Record is valid")

    if roll not in ids:
        ids.append(roll)
```

## 8. Test Cases

| Test / Input | Expected result |
|---|---|
| 101, Ravi, 85, 90: Unique ID; valid fields | Record is valid |
| 101, Arun, 95, 88: Roll 101 already exists | Duplicate ID |
| 104, blank name, 110, 75: Name missing; marks > 100 | Missing name; Invalid marks |

## 9. Sample Output Screenshot




The image above is an illustrative console-style sample output based on the listed example data.

## 10. Conclusion

Tuples keep each student's fields together, while lists and conditions help identify duplicate or inconsistent records.